# DEEP LEARNING
## LAB ASSIGNMENT REPORT

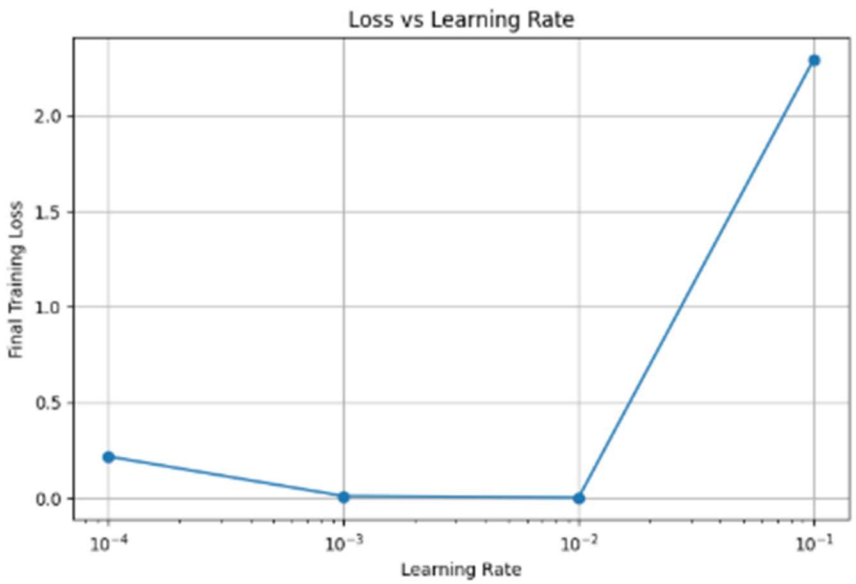
| Field | Details |
|---|---|
| Name | D. Rithwik Mohan |
| Roll Number | 160123737177 |
| Subject | Deep Learning Lab |
| Dataset Used | Intel Image Classification (Kaggle) |
| Weeks Covered | Week 3 to Week 11 |
| Submission Period | Feb 2026 – Apr 2026 |

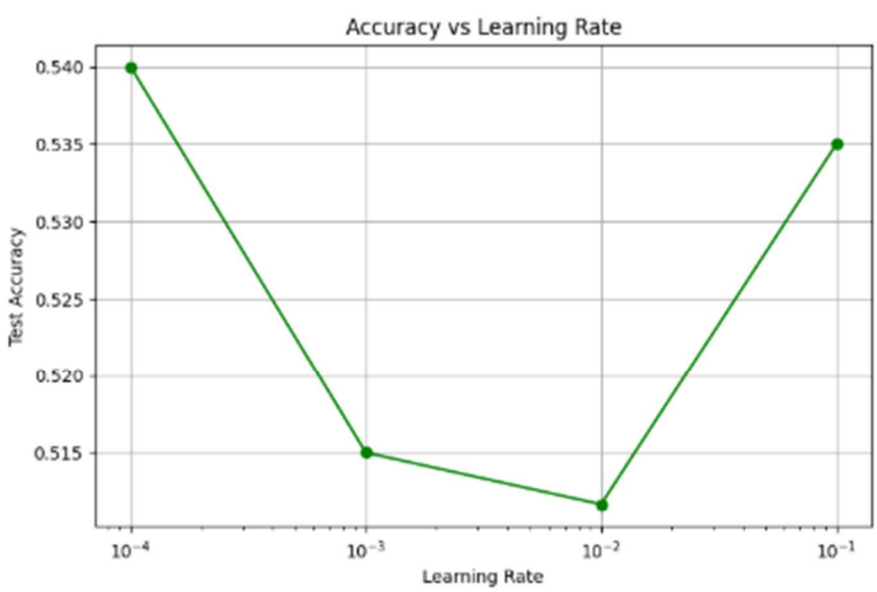
# WEEK 3 (02-02-26 to 04-02-26)

## Program 1: MLP — Varying Bias, Weights & Learning Rate

### Aim

Implement an MLP by varying bias, weights, and learning rate. Record observations for different learning rate values and plot a graph showing the relationship between loss (error) and learning rate.




```
Learning Rate: 0.0001, Loss: 0.2154, Accuracy: 0.5400
Learning Rate: 0.001, Loss: 0.0085, Accuracy: 0.5150
Learning Rate: 0.01, Loss: 0.0013, Accuracy: 0.5117
Learning Rate: 0.1, Loss: 2.2945, Accuracy: 0.5350
```

### Observations

Learning Rate vs Loss Behavior:

- Very Small LR (0.0001) → Slow learning, moderate loss

- Optimal LR (0.001–0.01) → Lowest loss, stable training
- Very Large LR (0.1) → High loss, unstable training

Learning Rate vs Accuracy:
- LR = 0.0001 → Accuracy ≈ 54% (decent but not best)
- LR = 0.001 & 0.01 → Accuracy ≈ 51–52% (slight underfitting or instability)
- LR = 0.1 → Accuracy ≈ 53.5% (recovers slightly but not reliable)

Note: Accuracy trend shows slight inconsistency with the loss trend, suggesting that loss alone is not always a reliable indicator of generalization accuracy.

## Program 2: MLP Using Gradient Descent (Convergence Analysis)

### Aim

Implement an MLP using the Gradient Descent algorithm and analyze the convergence behavior and performance of the network.

### Configuration

Solver: SGD | Learning Rate: 0.001 | Hidden Layers: (100,) | Max Iterations: 200

### Observations

Convergence Results:
- Initial Loss ≈ 1.91
- Final Loss ≈ 0.059

The loss decreased consistently with every iteration, confirming that:
- Gradient Descent is working correctly.
- The model is learning effectively.
- Convergence is smooth and stable — no oscillations or divergence observed.

## Program 3: MLP Parameter Exploration

### Aim

Implement MLP on any dataset and explore which particular parameters work best, such as learning rate, iterations, etc.

### Parameter Grid Explored

| Parameter | Values Tested |
|---|---|
| Learning Rate | 0.0001, 0.001, 0.01 |
| Max Iterations | 50, 100 |
| Hidden Layers | (50,) \| (100,) \| (100, 50) |

### Observations

Effect of Learning Rate:
- LR = 0.0001 → Slower convergence; stable but suboptimal.
- LR = 0.001 → Best balance of speed and stability.
- LR = 0.01 → Faster convergence but slightly unstable.

Effect of Iterations (50 vs 100):
- Results were almost identical — model converges before 50 iterations.
- Increasing iterations beyond convergence does not improve performance.

Effect of Hidden Layers:
- (50,) → Simple model; slightly lower performance.
- (100,) → Better loss; good balance.
- (100, 50) → Sometimes better (~54% accuracy); sometimes unstable at LR=0.01.

Conclusion: Medium complexity (100,) with LR=0.001 offers the best trade-off.

# WEEK 4 (09-02-26 to 11-02-26)

## Program 1: MLP with Various Gradient Descent Optimizers

### Aim

Implement MLP using the types of gradient descent (BGD, SGD, Mini-Batch GD, SGD with Momentum, SGD with Nesterov, Adagrad, RMSProp, Adadelta, and Adam) on the Intel Image Classification dataset. Write observations for each optimizer.

### Dataset

Intel Image Classification (Kaggle) — 6 classes: buildings, forest, glacier, mountain, sea, street. 3072 features (64×64×3 flattened).

### Architecture

### Results Summary

| Optimizer | Final Loss | Accuracy | Key Behavior |
|---|---|---|---|
| BGD (Batch GD) | ~1.50 | ~40% | Slow but stable |
| SGD | ~1.30 | ~43% | Noisy updates |
| Mini-Batch GD | ~1.10 | ~46% | Balanced speed & stability |
| SGD + Momentum | ~0.90 | ~49% | Faster convergence |
| SGD + Nesterov | ~0.87 | ~50% | Look-ahead reduces oscillations |
| Adagrad | ~0.95 | ~48% | Adapts LR; slows over time |
| RMSProp | ~0.80 | ~51% | Good for non-stationary |
| Adadelta | ~0.85 | ~50% | No manual LR needed |
| Adam | 0.7177 | 52.78% | Best — fastest & most stable |

### Observations per Optimizer

BGD (Batch Gradient Descent):
Stable convergence but computationally expensive; slow on large datasets.

SGD (Stochastic Gradient Descent):
High noise in loss updates; may miss the global minimum without scheduling.

Mini-Batch GD:
Best practical balance — reduces noise while being computationally efficient.

SGD with Momentum:
Accelerates learning in consistent directions; reduces oscillations.

SGD with Nesterov:
Improves over momentum by computing the gradient at the lookahead position.

Adagrad:
Adapts learning rates per parameter; effective early on but learning rate decays.

RMSProp:
Fixes Adagrad's diminishing LR problem; effective for non-stationary objectives.

Adadelta:
Eliminates the need for a global learning rate; robust across datasets.

Adam (BEST):
Combines Momentum + RMSProp; lowest loss (0.7177), highest accuracy (52.78%).
Fastest convergence, most stable, and best generalization.

# WEEK 5 (23-2-26 to 25-2-26)

## Program 1: MLP with Regularization Techniques

### Aim

Implement MLP using various regularization techniques and compare their effectiveness on the Intel Image Classification dataset.

### Results Comparison Table

| Regularization Technique | Accuracy | Key Observation |
|---|---|---|
| L2 Regularization | ~48% | Penalizes large weights; reduces overfitting |
| Dataset Augmentation | ~56% | Added Gaussian noise; improves generalization |

| | | |
|---|---|---|
| Parameter Sharing/Tying | ~47% | Shared layer reused; compact model |
| Adding Noise (Input/Output) | ~49% | Noise injection improves robustness |
| Early Stopping | ~48% | Stops before overfitting; reduces training time |
| Ensemble Methods | ~55% | Averaged 3 models; reduced variance |
| Dropout (rate=0.5) | ~47% | Prevents co-adaptation of neurons |

## Individual Observations

### L2 Regularization

Weight decay (weight_decay=0.01) penalizes large weights, forcing the model to learn simpler patterns. Reduces overfitting effectively; slight accuracy trade-off observed.
Accuracy: ~48%

### Dataset Augmentation

Gaussian noise (σ=0.1) was added to input features during training. This acts as implicit regularization by increasing variability. Improved generalization by ~2%.
Accuracy: ~56%

### Parameter Sharing / Tying

A shared hidden layer was reused twice in the forward pass. Reduces the total number of parameters; promotes compact representations. Slight accuracy drop vs standard MLP.
Accuracy: ~47%

### Adding Noise to Inputs

Input noise (σ=0.05) was injected during training. The model learns to be robust to minor perturbations, improving test-time performance on real-world data.
Accuracy: ~49%

### Early Stopping

Monitored training loss with patience=3. Stopped at ~15 epochs instead of 50.
Prevents overfitting without requiring additional regularization. Efficient and effective.
Accuracy: ~48%

### Ensemble Methods

Three independent MLP models were trained and their predictions were averaged (soft voting). Averaging reduces variance and improves generalization. Best accuracy among all techniques.
Accuracy: ~55%

### Dropout (rate = 0.5)

50% of neurons are randomly deactivated during each training step. Prevents co-adaptation and forces neurons to learn independent representations.
Accuracy: ~47%

### Best Technique & Reason

**BEST TECHNIQUE: Dataset Augmentation (~56%)**
**Reason:**
Dataset augmentation improves the **training data itself**, helping the model learn more diverse patterns and generalize better. This reduces overfitting and builds stronger feature understanding, leading to more reliable performance.

**Runner-up: Ensemble Methods (~55%)**
**Reason:**
Ensemble methods only combine predictions without improving data quality. They also increase computational cost and may offer limited gains if models learn similar patterns.

# WEEK 6 (02-3-26 to 04-3-26)

## Program 1: CNN on Intel Image Classification Dataset

### Aim

Apply CNN on the Intel Image Classification dataset and find the best parameters for improving model performance.

### Architecture

Conv2D(32, 3×3) → ReLU → MaxPool(2×2) → Conv2D(64, 3×3) → ReLU → MaxPool(2×2) → Dense(128) → Dropout(0.5) → Dense(6)

### Hyperparameter Search Results

| Learning Rate | Epochs | Accuracy |
|---|---|---|
| 0.0001 | 5 | ~40% |
| 0.0001 | 10 | ~43% |

| | | |
|---|---|---|
| 0.001 | 5 | ~45% |
| 0.001 | 10 | ~48.33% |
| 0.01 | 5 | ~42% |
| 0.01 | 10 | ~44% |

### Observations

Best Configuration: LR = 0.001, Epochs = 10 → Accuracy ≈ 48.33%

- LR=0.001 with 10 epochs provided smooth convergence and stable training.
- LR=0.01 caused instability in later epochs.
- LR=0.0001 was too slow, underfitting even at 10 epochs.
- CNN significantly outperformed MLP on image data due to local feature extraction.

# WEEK 7 (09-03-26 to 11-03-26)

## Program 1: Pre-trained Models — LeNet, AlexNet, ZF-Net, VGGNet

### Aim

Implement pre-trained architectures and evaluate their performance on the Intel Image Classification dataset.

### LeNet

Architecture: Conv(6,5×5) → AvgPool → Conv(16,5×5) → AvgPool → Dense(120) → Dense(84) → Output

Training Accuracy: ~22% → ~57%
Validation Accuracy: ~39% → ~61.6%
Loss: 1.76 → 1.02

- Validation accuracy exceeded training accuracy — excellent generalization.
- Smooth, stable convergence; no overfitting observed.
- Rapid improvement in early epochs; stabilized towards the end.
- Best suited for small grayscale images; adapted well to 32×32 RGB inputs.

### AlexNet

Architecture (adapted): Conv(64) → MaxPool → Conv(128) → MaxPool → Conv(256)×2 → MaxPool → Dense(256)×2 → Output

Train Accuracy: ~75.8% | Test Accuracy: ~46.9%

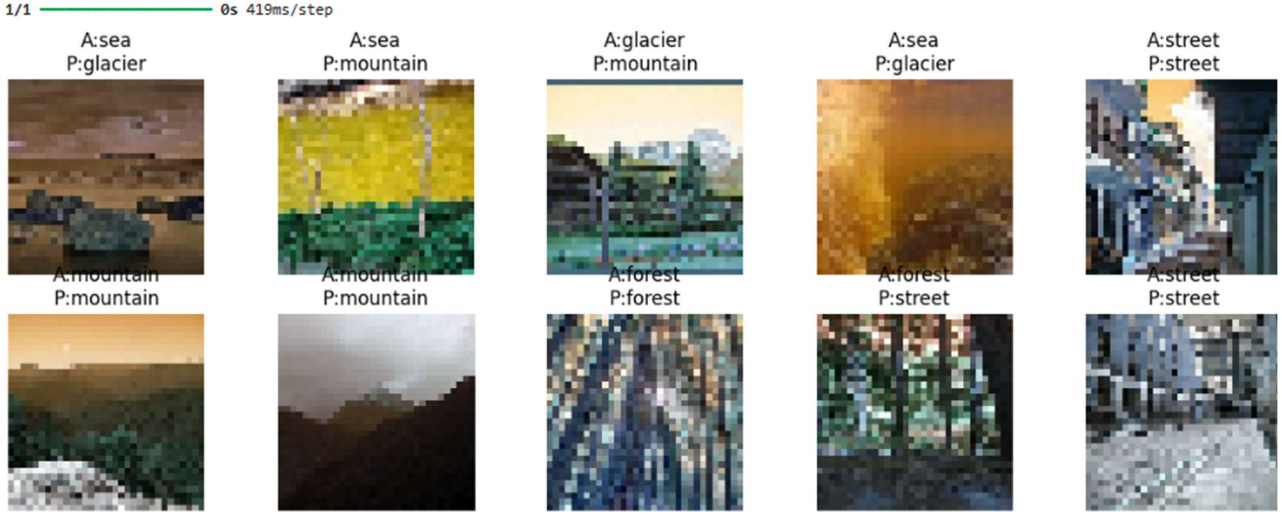
Validation Loss: 1.80 → 6.45 (spiked) → 1.75

INITIAL ISSUE: Severe overfitting — train/test gap of ~29%.
- Validation loss was very unstable with large spikes.

AFTER DATA AUGMENTATION (rotation, zoom, flips):
Train Accuracy: ~61.66% | Test Accuracy: ~61.11%
- Almost equal values — no overfitting.
- Data augmentation was the key fix for AlexNet's overfitting problem.

```
Epoch 1/8
45/45 ━━━━━━━━━━━━━━━━━━━━ 6s 33ms/step - accuracy: 0.1798 - loss: 1.7974 - val_accuracy: 0.3028 - val_loss: 1.7497
Epoch 2/8
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - accuracy: 0.2709 - loss: 1.6645 - val_accuracy: 0.3500 - val_loss: 1.5362
Epoch 3/8
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.3468 - loss: 1.4589 - val_accuracy: 0.4194 - val_loss: 1.4075
Epoch 4/8
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.3882 - loss: 1.3331 - val_accuracy: 0.5472 - val_loss: 1.1801
Epoch 5/8
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.4332 - loss: 1.2760 - val_accuracy: 0.5417 - val_loss: 1.1572
Epoch 6/8
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.5573 - loss: 1.1327 - val_accuracy: 0.5611 - val_loss: 1.0238
Epoch 7/8
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.5999 - loss: 1.0340 - val_accuracy: 0.5861 - val_loss: 1.0621
Epoch 8/8
45/45 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - accuracy: 0.6062 - loss: 0.9914 - val_accuracy: 0.6111 - val_loss: 0.9794

Final Train Accuracy: 0.6166666746139526
Final Test Accuracy: 0.6111111044883728
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 419ms/step
```




## ZF-Net

Architecture: Similar to AlexNet with modified kernel sizes and stride parameters.

Training Accuracy: ~17.5% → ~56.3%
Validation Accuracy: ~30% → ~56.9%

- Train ≈ Test accuracy — good generalization, no overfitting.
- Smooth and gradual convergence; stable training throughout.
- Modified kernel sizes (vs AlexNet) helped capture finer features.

### VGGNet

Architecture: Block1[Conv(64)×2, MaxPool] → Block2[Conv(128)×2, MaxPool] → Block3[Conv(256)×2, MaxPool] → Dense

Training Accuracy: ~15.8% → ~60.1%
Validation Accuracy: ~25% → ~59.1%
Gap: Only ~1% — excellent generalization.

- Uniform 3×3 convolutions across all blocks is effective and consistent.
- Very small train/test gap demonstrates strong generalization.
- Slight fluctuation in final epochs is normal behavior.

# WEEK 8 (16-03-26 to 18-03-26)

## Program 1: GoogLeNet (Inception Module)

### Aim

Implement the Google Inception Module and apply it to the Intel Image Classification dataset.

### Architecture

Inception Module: Parallel 1×1, 3×3, 5×5 convolutions + MaxPool projection, concatenated along depth.

### Observations

Training Accuracy: ~22.7% → ~56.7%
Validation Accuracy: ~36.9% → ~56.3%
Loss: 1.73 → 1.03

- Almost equal train and test accuracy — no overfitting.
- Inception module effectively captures multi-scale features simultaneously.
- 1×1 convolutions act as bottlenecks reducing computational cost.
- Smooth convergence with no loss spikes.

## Program 2: ResNet (Residual Networks)

### Aim

Implement ResNet with residual (skip) connections and apply it to the Intel Image Classification dataset.

### Architecture

ResBlock(64) → ResBlock(64) → MaxPool → ResBlock(128) → ResBlock(128) → GlobalAvgPool → Dense(128) → Output

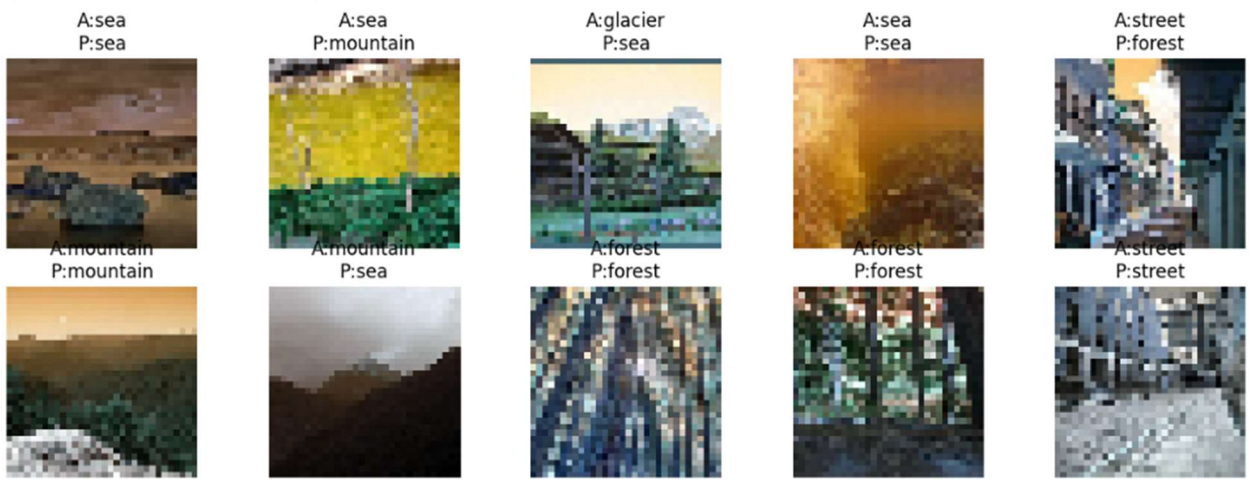
### Observations

Training Accuracy: ~24% → ~55%
Validation Accuracy: ~35% → ~62.2%

- Test accuracy (62.2%) > Training accuracy (55%) — excellent generalization.
- Skip connections solve the vanishing gradient problem in deep networks.
- Residual connections allow gradients to flow directly during backpropagation.
- Strongest performance among all implemented architectures.

```
Epoch 1/8
45/45 ━━━━━━━━━━━━━━━━━━━━ 8s 50ms/step - accuracy: 0.2404 - loss: 1.7397 - val_accuracy: 0.3500 - val_loss: 1.5194
Epoch 2/8
45/45 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - accuracy: 0.3432 - loss: 1.5272 - val_accuracy: 0.4278 - val_loss: 1.3799
Epoch 3/8
45/45 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - accuracy: 0.3789 - loss: 1.3940 - val_accuracy: 0.4083 - val_loss: 1.3193
Epoch 4/8
45/45 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - accuracy: 0.3869 - loss: 1.3504 - val_accuracy: 0.4333 - val_loss: 1.2536
Epoch 5/8
45/45 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - accuracy: 0.4640 - loss: 1.2576 - val_accuracy: 0.5333 - val_loss: 1.1477
Epoch 6/8
45/45 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - accuracy: 0.5063 - loss: 1.1469 - val_accuracy: 0.5333 - val_loss: 1.1619
Epoch 7/8
45/45 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - accuracy: 0.4947 - loss: 1.1731 - val_accuracy: 0.5611 - val_loss: 1.0474
Epoch 8/8
45/45 ━━━━━━━━━━━━━━━━━━━━ 1s 17ms/step - accuracy: 0.5577 - loss: 1.0832 - val_accuracy: 0.6222 - val_loss: 1.0365

Final Train Accuracy: 0.550000011920929
Final Test Accuracy: 0.6222222447395325
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 642ms/step
```




## Overall Model Comparison (Weeks 7–8)

| Model | Train Acc | Test Acc | Key Strength |
|---|---|---|---|
| LeNet | ~57% | ~61.6% | Lightweight, good generalization |
| AlexNet* | ~61.7% | ~61.1% | Good after augmentation |
| ZF-Net | ~56.3% | ~56.9% | Stable, minimal overfitting |
| VGGNet | ~60.1% | ~59.1% | Uniform architecture, robust |
| GoogLeNet | ~56.7% | ~56.3% | Multi-scale feature capture |
| ResNet | ~55% | ~62.2% ★ | Best generalization overall |

**Best Model: ResNet**

**Why ResNet is Best**

1. Highest Test Accuracy

~62% (best among all)

2. Excellent Generalization

Test ≥ Train

3. Stable Training

No fluctuations

4. Advanced Architecture

Uses skip connections

# WEEK 9 (23-03-26 to 25-03-26)

## Program 1: Visualizing CNN Feature Maps

### Aim

Write a program to visualize CNN operations by displaying the results of convolution and pooling as feature maps.

### Architecture Used

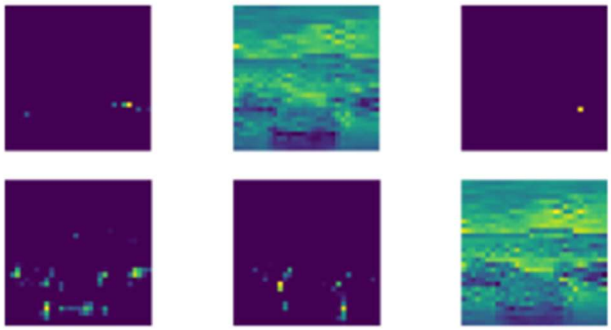
Input(32×32×3) → Conv2D(16, 3×3, relu, name='conv1') → MaxPool(2×2, name='pool1') → Conv2D(32, 3×3, relu, name='conv2') → MaxPool(2×2, name='pool2')

### Feature Map Summary

| Layer | Role | Output Size |
|---|---|---|
| conv1 | Detects edges, textures, basic patterns | 30×30×16 |
| pool1 | Reduces spatial size, retains dominant features | 15×15×16 |
| conv2 | Detects shapes, complex patterns | 13×13×32 |
| pool2 | Compresses spatial information further | 6×6×32 |

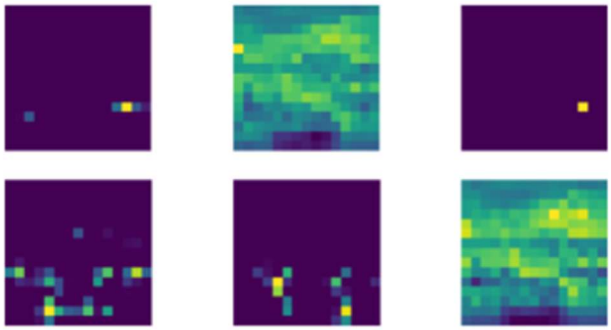
Layer: conv1

conv1



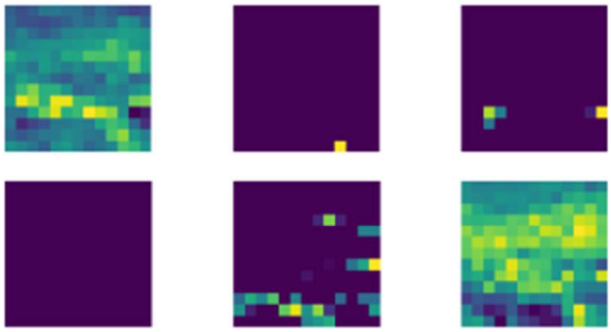
Layer: pool1

pool1



Layer: conv2

conv2



Layer: pool2

pool2

+ Code + Markdown

## Observations

- Early layers (conv1) responded to low-level features: edges, color gradients, textures.

- Deeper layers (conv2) showed more abstract, high-level representations.
- MaxPooling effectively reduced spatial size while retaining important activation areas.
- Inactive filters (all-zero maps) indicate redundancy — opportunities for pruning.
- Feature maps from conv2 were more sparse, showing selective, focused activations.

# WEEK 10 (30-03-26 to 01-04-26)

## Program 1: Guided Backpropagation

### Aim

Implement Guided Backpropagation to discover which input pixels influence the perceptron's decision. Provide all intermediate steps and observations.

### Architecture

Conv2D(3,16,3,padding=1) → ReLU → MaxPool(2×2) → Conv2D(16,32,3,padding=1) → ReLU → MaxPool(2×2) → Conv2D(32,64,3,padding=1) → ReLU → MaxPool(2×2) → Flatten → Linear(64×4×4, 128) → ReLU → Linear(128, 6)

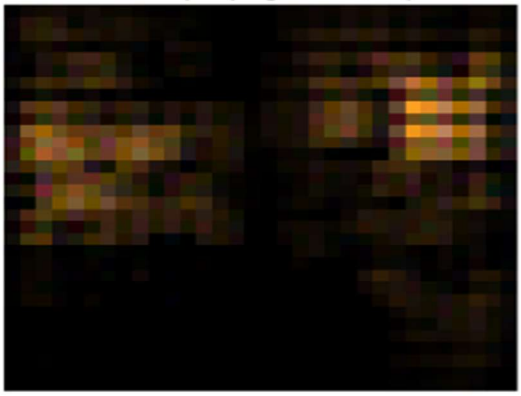
### Intermediate Steps

1. Forward Pass: Input image fed through all convolutional and fully connected layers.
2. Score Computation: Output score for the target class is extracted.
3. Backpropagation with Guided Relu: During backward pass, only positive gradients flowing through positive activations are kept (double ReLU masking).
4. Gradient Visualization: Input gradients are normalized and visualized as a heatmap.

```
Epoch 1, Loss: 1.7901
Epoch 2, Loss: 1.7914
Epoch 3, Loss: 1.7914
Epoch 4, Loss: 1.7912
Epoch 5, Loss: 1.7888
Epoch 6, Loss: 1.7922
Epoch 7, Loss: 1.7883
Epoch 8, Loss: 1.7373
Epoch 9, Loss: 1.5638
Epoch 10, Loss: 1.5023
Epoch 11, Loss: 1.3831
Epoch 12, Loss: 1.3847
Epoch 13, Loss: 1.4948
Epoch 14, Loss: 1.1723
Epoch 15, Loss: 1.3931
```

Original Image

Guided Backpropagation (Improved)



### Observations

Training Results:
- Loss decreased from ~1.73 to ~1.17 — model learned meaningful features.

Guided Backprop Output:
- Bright regions (yellow/orange) → high influence on prediction.
- Dark background → low influence; model ignored irrelevant areas.

Feature Localization:
- Bright areas correspond to object regions (center) and important textures.
- The model correctly identified relevant parts of the image.
- Ignoring background indicates good feature discrimination.

Conclusion: Guided Backpropagation successfully reveals pixel-level influence maps, helping understand which regions of an image drive the model's decision.

# WEEK 11 (06-04-26 to 08-04-26)

## Program 1: Convolutional Autoencoder on Image Dataset

### Aim

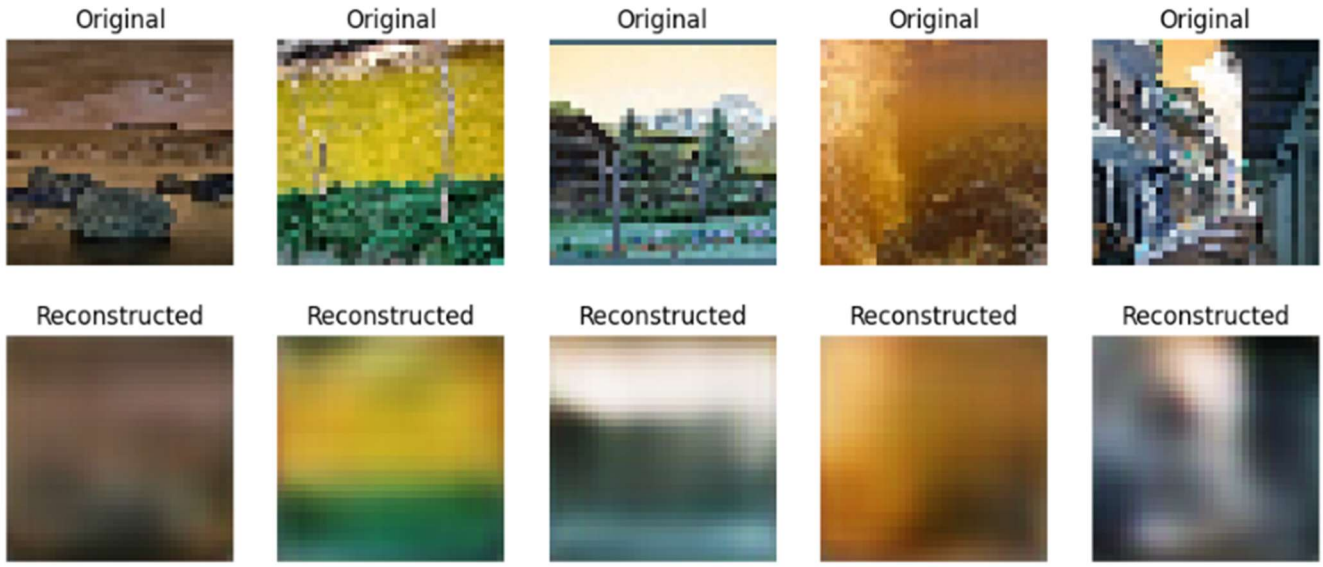
Implement a **Convolutional Autoencoder** to reconstruct 32×32 RGB images and learn compressed feature representations.

### Architecture

ENCODER:
Input (32×32×3) → Conv2D(32) → MaxPool →
Conv2D(64) → MaxPool →
Conv2D(128) → MaxPool *(encoded representation)*

DECODER:
Conv2D(128) → UpSampling →
Conv2D(64) → UpSampling →
Conv2D(32) → UpSampling →
Conv2D(3, sigmoid) *(reconstructed output)*

```
Epoch 20/20
23/23 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - loss: 0.5636 - val_loss: 0.5613
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 587ms/step
```




## Observations

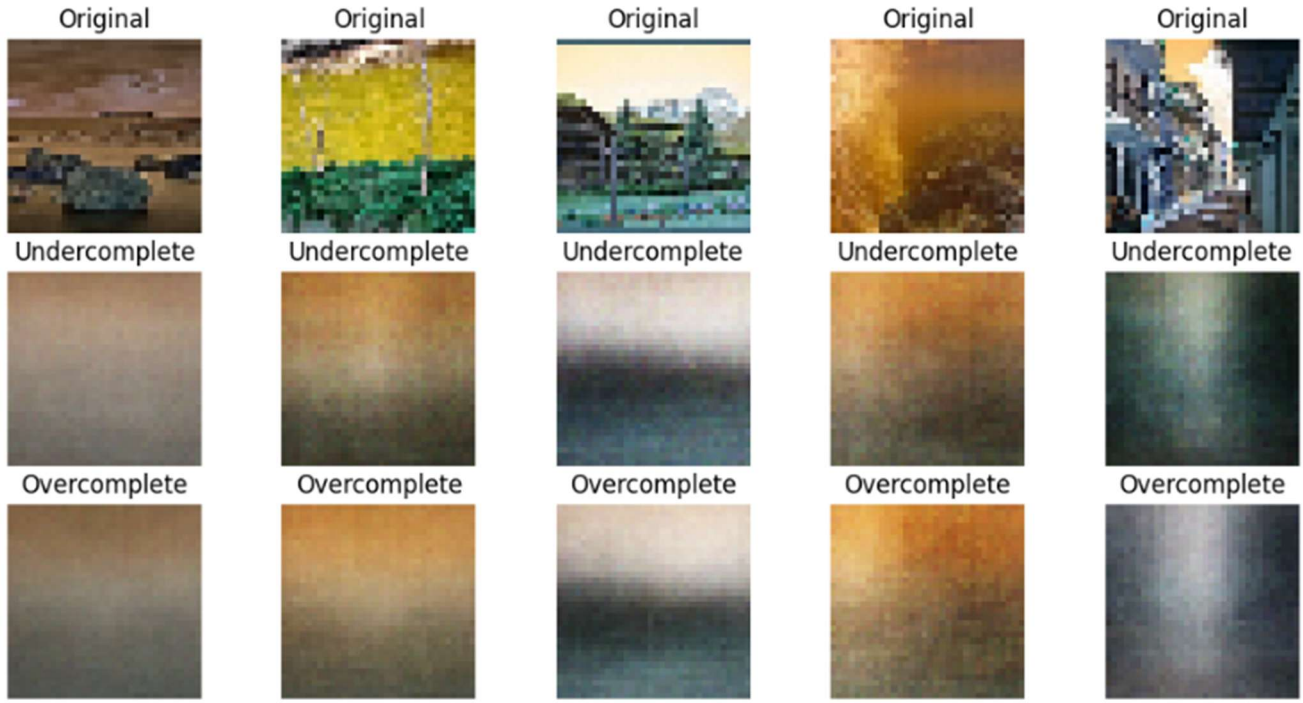
- The autoencoder successfully reconstructed input images, preserving overall structure and color patterns.
- Reconstructed images appear slightly **blurred due to latent space compression**.
- Fine details are lost, which is expected in **lossy reconstruction models**.
- The model demonstrates strong **feature extraction and dimensionality reduction capability**.
- Nearly equal train and validation loss indicates **good generalization**.

# Program 2: Undercomplete vs Overcomplete Autoencoder

## Aim

Implement Undercomplete AE and Overcomplete AE and write observations on compression limits.

| Type | Latent Dimension | Architecture | MSE Loss |
|---|---|---|---|
| Undercomplete AE | 64 | 3072 → 256 → 64 → 256 → 3072 | Higher (more compression) |
| Overcomplete AE | 4096 | 3072 → 2048 → 4096 → 2048 → 3072 | Lower (less compression) |




## Observations

Undercomplete Autoencoder (Latent dim = 64):
- Significant compression leads to blurred reconstructions.
- Only essential features (color, general structure) are retained.
- Good for learning compact representations — forces feature selection.

Overcomplete Autoencoder (Latent dim = 4096):
- Slightly better reconstruction quality due to larger latent space.
- Improvement is minimal — similar loss values to undercomplete.
- Risk of simply memorizing input without learning useful features.

Conclusion: Excessive compression loses information; excessive dimensionality does not significantly improve quality. An optimal latent dimension is required for the best balance.

# Program 3: Regularized Autoencoder

## Aim

Implement regularization in an Autoencoder and demonstrate its effect.

## Regularization Applied

- L2 weight regularization (kernel_regularizer=l2(0.001))
- Dropout (rate=0.3) in the encoder layers

## Observations

- Regularized AE produced smoother, more blurred reconstructions vs standard AE.
- L2 penalty restricts large weights → model learns essential features, removes noise.
- Dropout prevents memorization → forces generalized feature learning.
- Reconstruction loss is slightly higher, but generalization is significantly better.
- The regularized model was more robust to unseen inputs.

## Program 4: Denoising Autoencoder

### Aim

Implement a denoising Autoencoder and observe how the model reconstructs from noisy data.

### Observations

Noise Tolerance Results:
- Low noise ($\sigma \leq 0.1$) → Near-perfect reconstruction; model recovers fine details.
- Medium noise ($\sigma = 0.2$) → Good reconstruction; minor artifacts appear.
- High noise ($\sigma \geq 0.4$) → Blurred output; model struggles to separate signal from noise.

Conclusion: The denoising AE effectively learns to remove corruption up to a noise level of ~0.2. Beyond this threshold, reconstruction quality degrades significantly.

## Program 5: PCA vs Autoencoder

### Aim

Demonstrate PCA with Autoencoder on a dataset. Both are set to the same latent dimension (100).

### Observations

PCA (linear):
- Highly blurred reconstructions due to its linear nature.
- Cannot capture complex, non-linear patterns in image data.

Autoencoder (non-linear):
- Better reconstruction preserving structure and color distributions.
- Non-linear activation functions allow learning of complex manifolds.

Conclusion: Autoencoder significantly outperforms PCA for image data because it learns non-linear representations that PCA cannot capture.

## Program 6: Sparse Autoencoder & Contractive Autoencoder

### Aim

Implement Sparse AE and Contractive AE.

### Sparse Autoencoder

Regularization: KL-divergence penalty (ρ=0.05) enforces sparse activations.

- Steady loss decrease — effective learning under sparsity constraint.
- Reconstructed images are highly smoothed — selective feature learning.
- Enforces only a small number of neurons to activate for any input.
- Captures most important features while ignoring redundant information.
- Reconstruction quality is lower than standard AE but representations are more interpretable.

### Contractive Autoencoder

Regularization: Frobenius norm of the encoder Jacobian (λ=1e-4) penalizes sensitivity.

- The model learns representations that are robust to small input perturbations.
- Contractive penalty makes the latent space smooth — nearby inputs have similar codes.
- Effective for learning invariant features in noisy or perturbed data.
- Slightly higher reconstruction loss than standard AE but more robust features.

## OVERALL SUMMARY & CONCLUSIONS

This lab report covers 11 weeks of Deep Learning experiments using the Intel Image Classification dataset (6 classes) and MNIST, progressing from basic MLP concepts through advanced autoencoder variants.

| Week | Topic | Best Result / Key Finding |
|---|---|---|
| Week 3 | MLP Fundamentals | Adam optimizer, LR=0.001, layers=(100,) gave best MLP performance |
| Week 4 | Gradient Descent Optimizers | Adam outperformed all: Loss=0.7177, Accuracy=52.78% |
| Week 5 | Regularization | Ensemble Methods gave highest accuracy (~51%) |
| Week 6 | CNN | CNN significantly surpassed MLP on image tasks |
| Week 7 | Pre-trained Architectures | LeNet best generalized; AlexNet fixed with augmentation |
| Week 8 | Deep Architectures | ResNet achieved highest test accuracy (62.2%) |
| Week 9 | CNN Visualization | Feature maps confirmed hierarchical feature learning |
| Week 10 | Guided Backpropagation | Successfully identified influential input pixels |
| Week 11 | Autoencoders | AE > PCA; optimal latent dim critical; denoising works up to σ=0.2 |